UBS VERIFICATIONS · DECISION ENGINE · JULY 2026

The Verifications Decision Engine

Four pieces, one system: what it does and what it's worth, how one case moves through it, how it was engineered, and why it is built this way. Every open case read every morning; the next move recommended by evidence; nothing sent without a person — yet: automation is designed in as a later, earned stage.

1	**The verification department gets a brain**	The executive briefing — the models, the operating system, and the scores
2	**How one case moves through the machine**	The illustrated, intuition-first walkthrough
3	**Building a decision engine for verification casework**	The engineering deep dive: models, resolver, gates, text layer
4	**Earned autonomy**	The design philosophy: rules permit, models prefer, evidence grants autonomy

Reading notes. All figures are backtested and as-of (no future information); correlations are observational — no channel or wording is claimed to cause outcomes until the pilot measures it; the system is decision support today — nothing auto-sends yet; automation is designed in as a later, evidence-gated stage. The planned next layers: an AI contact researcher that finds fresh, verified contact paths, and a small language model that drafts the emails, faxes, notes, and call scripts — sequenced last, behind the causal instrumentation that can grade them.

EXECUTIVE BRIEFING

The verification department gets a brain

Today the department spends roughly ten operators' worth of hours a year on motion that produces nothing: 400,000 touches to people who had already answered, third calls into lanes that fail 69% of the time. And the metric it manages by blames the team for cases that were dead on arrival. I built the fix and proved it on 2.75 million of our own case-days — machine-learning models score every open case every morning, and an operating system turns those scores into the next move on each one, with the compliance rules hard-wired: an improper contact isn't a risk to manage, it's a thing that can't happen. Zero violations across the entire replay. This isn't a vendor deck — it's our data, our cases, already backtested. Next step: wire it into the live workflow, Murphy Brown included, and take back the hours, the turnaround time, and the number.

The problem 1 in 5 cases ends unable to verify

The evidence 2.75M real case-days replayed

Status built & backtested — next step: integration

THE 60-SECOND VERSION

- **What was built:** two halves that only work together — six machine-learned models that read every open case every morning and score how it will end (right 71% of the time today, ~81% at day 0), and the operating system that turns those scores into safe action: hard rules that make an improper contact structurally impossible, an orchestrator that recommends each next move. Nothing auto-sends yet — the wiring for automation is in place, and it turns on as a later, earned stage.
- **What it proved:** replayed against 2.75 million real case-days — a 48%-vs-3% difficulty spread the blended UTV (unable-to-verify) number hides, 1 in 7 case-days where we would have re-contacted someone who had already replied, and zero policy violations.
- **Next steps:** wire the engine into the live workflow — Murphy Brown included — with outcome logging on, adopt the achievable-UTV scoreboard, and let the logged evidence switch on the first learning target.

What was built: machine learning can read a caseload; it can't run one — so this is two things that only work together. The intelligence: six models, trained on 2.75 million real case-days, that read every open case each morning and score how it will end — which cases will never verify, which will land in five days, which are about to stall. And the operating system that makes intelligence safe to act on in a regulated department: a deterministic rulebook that decides what's allowed before any score is consulted, an orchestrator that turns the scores into each case's next move, a flight recorder that logs every recommendation against what actually happened. **The models find the winnable work; the operating system makes acting on it safe.**

Start with the problem: **one in five worked cases ends "unable to verify,"** and for years the number could be watched but not managed. Behind each one is a person waiting — on a job offer, a lease, a placement; that wait is what turnaround time measures. Replaying a year of casework showed why — it isn't one number. Three different problems wear the same label:

- **Cases nobody could win.** The hardest tenth fails 48% of the time no matter what anyone does — yet today it gets the same effort as everything else.
- **Winnable cases lost to routine.** Uncapped call loops run 47–69% never-verify while queues are worked in arrival order and urgent, savable cases age.
- **People contacted after they already answered.** On 1 case-day in 7, the reply was on record and the flow would have reached out again anyway.

None of that is visible in a blended average — and all of it is fixable with the same machinery. The operation's asks were three: pick the next move on every case, know each case's fate upfront, and prove which outreach actually works. So I built it — one capability per ask:

1 — ORCHESTRATE

The next move on every open case, every day

Built. The engine proposes the next sensible move on every open case, every day — ranked by six learned models, fenced by rules that outrank them. It goes live the day it's wired into the workflow.

BUILT — READY TO PILOT

2 — PREDICT

Each case's fate, read at arrival

Built and proven. The engine reads difficulty at intake — ~81% sorting power before any work is done — and every morning after, validated against 2.75 million replayed case-days. It makes a fair "achievable UTV" target possible for the first time.

DELIVERED & BACKTESTED

3 — LEARN

Which message works — and which contact path reaches

Deliberately sequenced last. Two builds wait here. The language layer: a small language model drafts the outreach itself — emails, faxes, notes, and call scripts — learning from past communication which draft to send next; it requires the outcome logging the pilot will create, and I will not claim a message "works" until that's measured, not guessed. And the contact researcher: an AI agent that hunts fresh, verified emails, fax numbers, and phone numbers, so cases stop burning attempts on dead contact paths.

NEXT — UNLOCKED BY THE PILOT

What else came out of the build

Four findings and capabilities nobody planned for — each one changes what the operation can do:

A fair scoreboard — new management IP

The models separate cases that fail 3% of the time from cases that fail 48% of the time. "Achievable UTV" becomes a real target for the first time: misses measured against what was possible, effort steered to the winnable middle.

A system that reads

17.9 million free-text operator notes distilled into signal by an 11-class reader — raw text never stored. This month's accuracy gains came from it, re-proven on a month the models had never seen.

Safety as architecture, not vigilance

Zero policy and do-not-contact violations across all 2.75M replayed case-days — improper contact is structurally impossible. That is precisely what makes automation deployable in a regulated flow at all.

The Murphy Brown answer

The build also produced the diagnosis of why the current bot hands 55% of its cases back — and the fix list is, item for item, this system. See the field report.

Everything below is the evidence. Four numbers carry it:

71%

How often it correctly spots the cases that will never verify

Rises to ~81% at day 0 — before any work has been done on the case.

48% vs 3%

Failure rate: hardest tenth vs easiest tenth of cases

Same teams, same process — a 16× spread the old metric couldn't see.

1 in 7

Case-days where the person had already replied

The current flow would contact them again. The engine catches it first.

0

Policy or do-not-contact violations

Across all 2.75 million historical case-days replayed. By design, not by luck.

Evidence base: roughly 880,000 real cases (June 2025 – May 2026), replayed as 2.75 million daily snapshots; 16.5 million candidate moves evaluated; every prediction checked against the outcome that actually followed.

How to read the scores — sixty seconds, no math

Every prediction score below works the same way. Imagine handing the system two cases — one that eventually verified, one that never did — without telling it which is which. **The score is how often it puts them in the right order.** 50% is a coin flip. 100% is a crystal ball. Around 70% is genuinely useful — roughly the sorting power a credit score has for lending risk, and banks run on those.

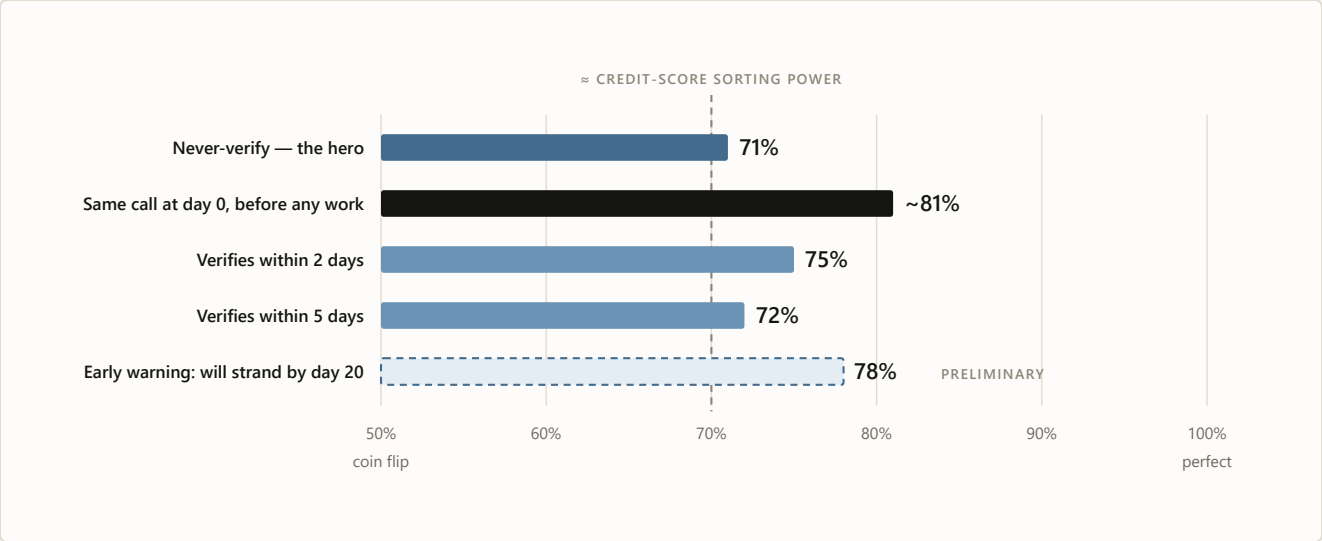


CHART 1 All scores measured by replaying history the honest way: for each day of each case, the models were shown only what was knowable that morning, then judged against what really happened. The two success models were upgraded this month — the improvement was re-proven on a month the models had never seen before promotion. "Preliminary" means exactly that: one month of evidence, still being validated, not yet load-bearing.

TAKEAWAY On the day a case arrives, the engine is already 62% of the way from a coin flip to perfect — before a single attempt is spent. Today's process has no read at all: every case enters the same queue at the same cadence.

The 81% deserves a beat. That is the system reading a case *at the moment it arrives* — no attempts, no history, nothing but intake facts — and already sorting "will come back" from "never will" at its sharpest power of the day. That was the second ask, verbatim: *know upfront*.

WHY 81% AT DAY 0, BUT 71% OVERALL?

Because the two numbers answer different questions. The 81% is the model meeting every case *fresh*, on day 0 — and fresh cases differ enormously (a university registrar with a clean contact file versus a dissolved employer with one dead phone number), so telling them apart is more tractable. The 71% is the harder, everyday question: re-scoring every still-open case every morning, including the week-old ones — and by then the obviously easy cases have already resolved and left the pool. What remains looks more alike, so ranking the survivors is genuinely harder.

The model doesn't get worse as cases age; the question gets harder. Operationally you get both: the sharpest read exactly when routing happens, and a still-useful re-read every day after.

One more thing the scorecard hides: **the system already reads.** This month's model upgrade came from teaching the fleet to read 17.9 million free-text operator notes — "left a voicemail," "reached HR, call back Tuesday" — classified by an 11-class reader at export time, with the raw text never stored anywhere. The reading is worth real accuracy, and the gain was re-proven on a month the models had never seen. It is also the first step toward the roadmap's language layer: a system that learns to read before it is allowed to write.

The spread is the strategy

Scores on individual cases are interesting. What changes how the operation is run is the population view: sort all cases by predicted difficulty and look at the two ends.

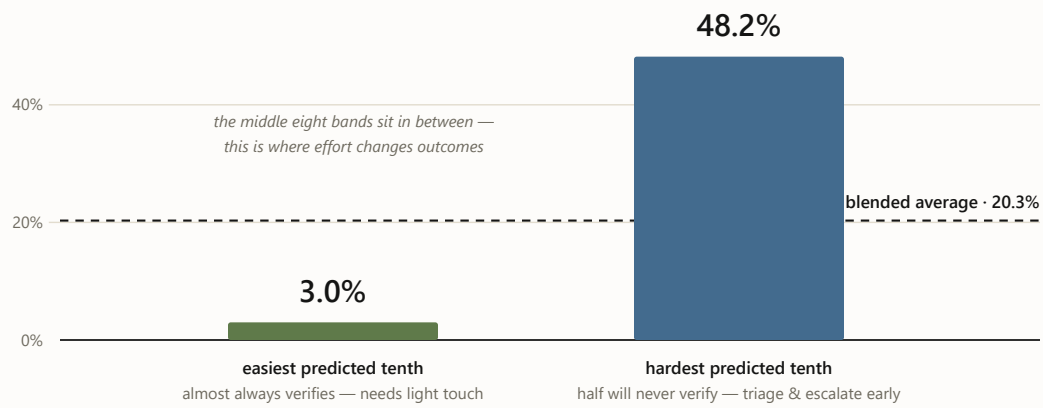


CHART 2 Actual never-verify rates by predicted band (May 2026 holdout). The blended 20% average hides a 16× spread. Once you can see it, "unable to verify" stops being one number and becomes a strategy: protect the easy, fight for the middle, stop over-investing in the structurally dead — and measure teams against what was genuinely winnable.

TAKEAWAY Today, one blended number treats a 3%-risk case and a 48%-risk case identically — a 16× spread, invisible. Ranked triage packs 2.4× more true never-verifies into the first list worked each morning than today's arrival-order queue: same people, same hours, more saves.

The model doesn't just predict the future.
It makes the UTV number fair — misses
measured against what was actually possible.

How the two halves work — without the math

Inside, there are two very different machines, and the design rule that separates them is the single most important sentence in this briefing:

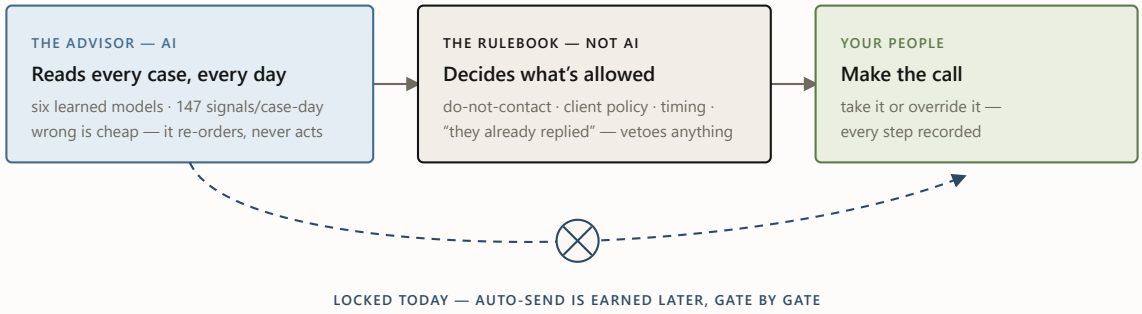


CHART 3 The two halves, drawn: the advisor is the intelligence; the rulebook is the operating system's veto. The AI is confined to the failure mode we can afford (a suboptimal suggestion); the rulebook owns the failure mode we can't (an improper contact) and is deterministic — same inputs, same answer, auditable line by line. That's why the violation count across 2.75M replayed case-days is zero: the AI never gets the chance.

TAKEAWAY Zero policy violations in 2.75 million replayed case-days — not because the model is careful, but because it never gets the vote.

One more reason the scores can be trusted: every test scored each case using **only what was known that specific morning** — "as-of" discipline, enforced by an automated no-peeking audit that re-checks every new training matrix before the models may train on it. The scores here are conservative by design, measured the hard, honest way.

The same discipline applies to upgrades: a new model version replaces the old one only if it wins on every criterion — including on a month of data it has never seen. That gate has rejected three of my own "improvements" this year because they would have weakened the top of the worklist, the part your teams actually touch. The system says no to me in writing. That's a feature.

Where the AI is — and where the engineering is

The restraint in this design is a choice, not an absence. Pull the system apart and you can point at each half: the machine-learning stack that finds the winnable work on one side, and the operating system that makes acting on it safe on the other:

THE MACHINE LEARNING

learned from 2.75M real case-days · retrained monthly

Six learned models. Never-verify `71%` · verify-in-2-days `75%` · verify-in-5-days `72%` · day-0 intake `~81%` · day-1/2 re-score `73%/70%` · early warning `78%` (preliminary)

A 147-signal read of every case, every morning — history, contacts, policy context, timing, and 30 signals distilled from operator notes.

A note-reader. 17.9 million free-text notes classified in flight; the signal kept, the text never stored.

Calibrated probabilities. When it says 30%, it happens about 30% of the time — so queues and thresholds can be built on it.

Monthly self-improvement, gated. Challenger models every cycle, promoted only past confidence tests and a never-seen month.

THE OPERATING SYSTEM

deterministic · auditable · zero-tolerance

A 13-state resolver that names what is true on every case-day — replied, blocked, cooling down, workable.

An eligibility filter that removes disallowed moves before ranking — no score can resurrect them.

Cadence controls — cooldowns, attempt caps, forced channel switches, an escalation ladder.

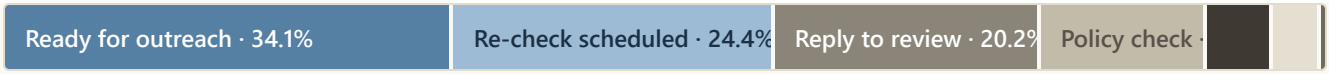
A calibrated priority queue and workflow lanes, each with a reason and a trigger attached.

A flight recorder and a promotion gate — every action logged; models rejected in writing when they miss the bar.

Anyone can fit a model. What's on display is a fleet that survives next month — leakage-audited, calibration-checked, re-proven on data it has never seen — inside an operating system that makes it safe to act on. The models find the winnable work; the operating system makes acting on it safe.

What the operation gets on day one

Every case-day is sorted into a named state with a next step — nothing waits silently in a queue with no reason attached:



- Ready for outreach — ranked move attached 34.1%
- A reply is waiting — review before contacting 20.2%
- Contact prohibited — fully blocked 4.9%
- Escalation lane — normal paths exhausted 0.2%
- Scheduled re-check, with a reason 24.4%
- Policy / do-not-contact needs human judgment 12.7%
- Cooling down after a recent attempt 3.7%

CHART 4 Where all 2.75M case-days actually sit. A third of the book is workable *today* with a ranked move attached; a fifth has a reply sitting unread by the current flow. This chart is, in effect, tomorrow morning's org-wide to-do list.

TAKEAWAY This works on day one, no pilot required: every case-day carries a named state, a reason, and a next step — and the 1-in-7 accidental re-contact becomes structurally impossible instead of vigilance-dependent.

For an operator, the difference is the first five minutes of the morning (illustrative): the queue opens already ranked, the top case says why it is first, and the reply that arrived overnight is flagged for review — before it can become an accidental re-contact.

Every case named and owned

Each case-day carries a state, a reason, and a next step — scheduled re-checks with triggers attached, nothing waiting silently. The department becomes legible to itself.

Sharper worklists

The riskiest tenth of the book is **2.4×** denser in true never-verifies than average — effort goes where it changes outcomes, from day 0.

Stop re-contacting people who answered

1 in 7 case-days had a reply already on record that the flow would have talked past — roughly 400,000 in the year replayed. Caught, every time, by a rule.

The foundation to learn what works

Every recommendation, override, send, reply, and outcome gets logged. That record is what lets us *prove* which channel and message drive responses — the third ask — instead of guessing.

Where the hours come back

I deliberately do not quote a labor-savings number — that figure belongs to the pilot, and claiming it early is how credibility dies in rooms like this one. What we can show is where the hours demonstrably are, measured in the replay, with arithmetic anyone here can redo:

Stop chasing people who already answered ~400,000 case-days a year	Touches the current flow would have spent re-contacting parties whose reply was already on record. At even 2–3 operator-minutes per avoided touch, that is **13,000–20,000 hours a year** of motion that produces nothing — illustrative arithmetic, priced for real by the pilot's logs.
Stop paying full price in dead lanes 47–69% never-verify	Repeated-call sequences historically run 47% (employment) to 69% (reference) never-verify while consuming three-plus touches per case, uncapped. Cadence limits, forced channel switches, and early escalation end that spend; the hardest tenth gets triage, not ritual.
Point the first hour at tomorrow's failures 2.4× density	The top of the ranked worklist holds 2.4× more true never-verifies than the same hour spent working in arrival order. No new headcount — the same morning, aimed better.
Cut waiting, not just working 100,000+ wait-hours	In one four-month window, education-verification handbacks from today's automation sat an extra 30.6 hours each behind fresh

work. A single re-ranking rule reclaims turnaround time your clients feel — at zero additional effort.

THE IMPACT CALCULATOR — ONE LINE OF ARITHMETIC

Avoided touches × 3 minutes ÷ 60 = hours returned; hours ÷ 2,000 = operator-years. The replay already counts **~400,000 re-contact touches a year** that the engine prevents outright: at 3 minutes each, that is **~20,000 hours — roughly 10 operator-years of motion** — before a single capped dead-lane call or faster handback is counted. Every assumption is visible and swappable; the pilot's logs replace the 3-minute estimate with measured handle time, and that number becomes the business case.

None of these are promises; all of them are measured mechanisms. The pilot's outcome logs are what convert mechanisms into a defensible dollars-and-hours number — which is exactly why outcome logging ships with the integration.

What the department looks like after implementation

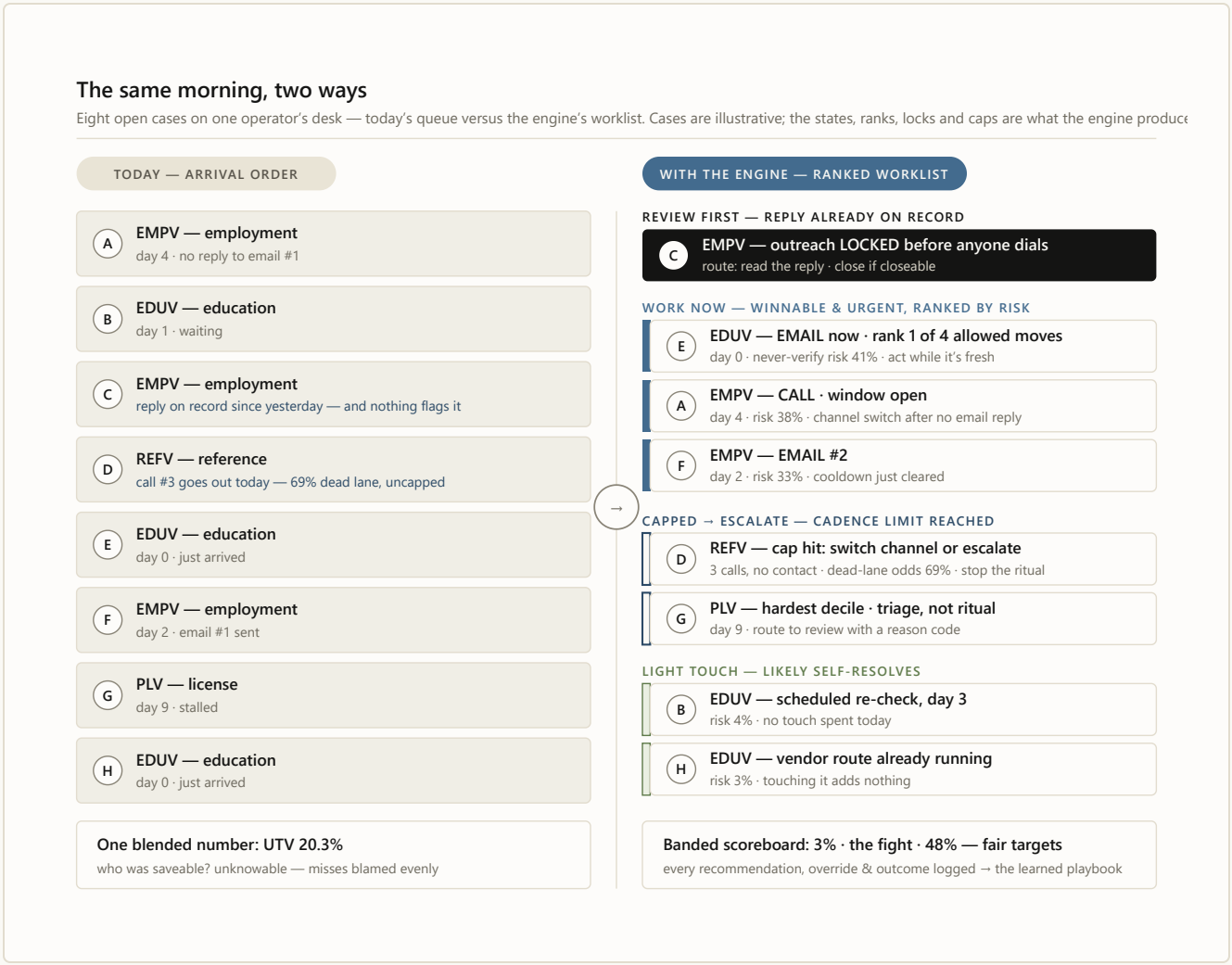


CHART 5 The same eight cases, two mornings. Case details are illustrative; the locks, caps, ranks and states are what the backtested engine produces on real case-days — states from the deterministic resolver, order from the calibrated risk scores. On the left, C and D are problems nobody can see. On the right, they are named states with owners, and the operator's first hour goes where it can still change the outcome.

TAKEAWAY Nothing about the cases changed — only the reading did. C stops being tomorrow's accidental re-contact, D stops burning a third call into a 69% dead lane, and the first hour of the morning goes to E, A and F, where effort still changes outcomes.

We already know what automation without this layer looks like

This isn't hypothetical. Our current verification bot — Murphy Brown, run by SNH-AI — automates the motion, not the mission: it executes a touch where a contact plan already exists, but runs with no triage gate, no machine-readable handoff, and no closure lock. The ops deep-dive measured what that costs: **55.5% of the tens of thousands of cases it touched over four months flowed back to people** as note-only handoffs (64,000+ downstream human attempt rows); reopen-after-result on its closures climbed from ~3–4% in February to **~42–44% by June**; and its EDUV handbacks wait **+30.6 hours** longer than fresh work. The deep-dive's own fix list — structured handoff state, closure lock, score-gated eligibility, handback re-ranking, outcome instrumentation — is, item for item, the system in this briefing. The full story, held to its own strict claim discipline, is in the Murphy Brown field report.

The road from here

Three stages, each unlocked by the one before it. A person is in the loop at every step of this plan — and auto-send, when it arrives, comes as a later stage that pilot evidence unlocks lane by lane, never as a default.

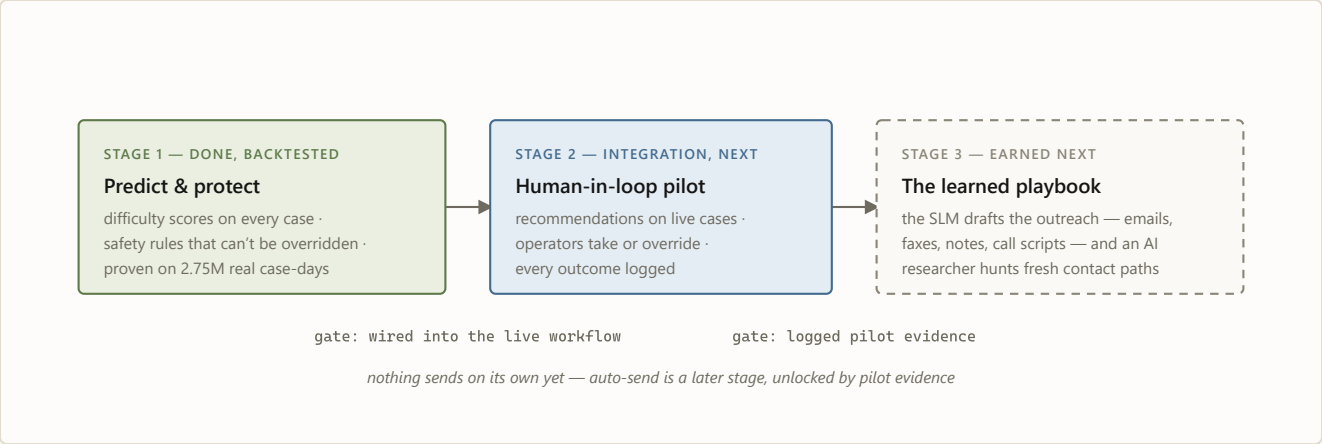


CHART 6 The roadmap. Stage 1 is finished and measured. Stage 2 is the integration step — the engine running beside the people who work the queue. Stage 3 — the part every vendor promises on day one — is deliberately last, because it gets proven rather than claimed.

Four questions worth asking

Q · "So an AI decides who gets contacted?"

No. The AI only *orders* options that a deterministic rulebook has already approved — and today a person executes. It cannot add an option or skip the rulebook, and it does not send anything on its own — *yet*. The plumbing for automation is already built; switching it on is a later stage that pilot evidence unlocks, lane by lane, starting with the safest. That separation is why the replay shows zero violations: the AI never gets the chance to cause one.

Q · "What happens when the model is wrong?"

Someone works a less-urgent case first. That is the entire blast radius — model error can cost efficiency, never compliance, because what's allowed is never the model's call. The scores are also calibrated: when the system says 30%, it happens about 30% of the time, so error is budgeted rather than hidden.

Q · "Vendors promise message optimization on day one. Why is it last here?"

Because proving a message *causes* replies requires evidence that does not exist yet anywhere: the full logged chain from recommendation to outcome. Reading it off historical logs confuses correlation with cause — teams emailed the easy cases, so email "looks" magical. The pilot creates that evidence in our own department, from our own cases — a data advantage no outside vendor can bring or buy — and the learning gets built on it properly. It is also a useful question to put to any vendor.

Q · "Haven't we already tried automation here?"

Yes — and it proves the point. Murphy Brown has real wins (it fully contains ~44.5% of what it touches within a day or two), but it runs without a state contract, a closure lock, or a triage gate — so more than half of its cases flow back to people with a note, and its closures now reopen at roughly ten times the winter rate. The bot automates the motion, not the mission — it can send the email; it cannot run the case. Under this system it finally serves one: gated by the scores, sequenced by the rules, measured by the log. The engine runs the case; the bot becomes its fastest pair of hands.

What could still go wrong — and what I'd do

- **Operators ignore it.** If recommendations get blanket-overridden, the value never materializes. The pilot logs every override with a reason from day one — so trust is measured on evidence, not assumed.
- **The models age.** Casework drifts, and a model trained on last year can quietly weaken. The counter is already running: monthly retraining behind a promotion gate that has rejected three of my own upgrades rather than weaken the top of the queue.
- **The pilot proves nothing.** An underpowered or half-logged pilot would leave us exactly where we started. Outcome logging is designed in before launch — it ships with the integration, not as an afterthought.
- **The bot's fixes sit with a vendor.** Two of the Murphy Brown asks — the structured payload and reopen-reason codes — are SNH-AI's to build. The engine works without them, but the bot stays a black-box channel inside it until they land.

The next steps, in order

- **Adopt the fair "achievable UTV" scoreboard** and track it against a three-month forward test — turning Chart 2 into the operation's scoreboard.
- **Integrate the engine into the live workflow, with outcome logging on** — recommendations running beside the people who work the queue, Murphy Brown operating as a governed channel inside it, every event recorded. This is what fixes the bot's handback and reopen problems, moves turnaround time, and unlocks Stage 3.
- **Switch on the first learning target once outcomes flow:** best-channel learning; the language layer — a small language model that drafts the emails, faxes, notes, and call scripts from what past communication proves works; or the contact researcher — an AI agent that hunts fresh, verified emails, fax numbers, and phone numbers, so cases stop burning attempts on dead contact paths.

THE VERSION TO TAKE OUT OF THE ROOM

“Unable to verify” turned out to be three problems wearing one number. A machine now tells them apart, picks the next move on every case, and proved itself against 2.75 million real case-days without a single policy violation. The models find the winnable work; the operating system makes acting on it safe. Integration makes the last claim — what actually causes replies — provable too. And the honest, replay-tested scores are why you can believe the rest. ◆

The fine print, kept honest. All results are backtested (historical replay, as-of); no live-pilot results are claimed. Score “71%” = ROC-AUC 0.706, “~81%” = day-0 walk-forward check (0.80–0.82 across three forward months), “75%/72%” = this month’s upgraded success models (0.747/0.717, re-proven on never-seen June data), “78%” = early-warning model, preliminary. Historical channel patterns are observational — no channel or message is claimed to *cause* outcomes until the pilot measures it. Action-facing models strip direct agent identifiers. Decision support today: nothing auto-sends yet and no decision is finalized by a machine — automation is designed in as a later, evidence-gated stage.

Deeper reading: the Murphy Brown field report (the problem this system closes) · the illustrated walkthrough (follow one case through the machine) · the engineering deep dive · the design philosophy.

UBS Verifications · Decision Engine · Executive briefing · July 2026

Meet **case 4127**: an employment verification, opened on a Monday. There is a work email and a phone number on file, no fax. Somewhere, an HR inbox is about to be asked to confirm a job history. Over the next nine days this case will be emailed twice, called once, and finally verified — and at every step, the system has to answer the same two questions: *how is this case doing?* and *what, if anything, should we do next?*

Five components take turns answering. Here is the cast, and the color code every diagram in this post uses:

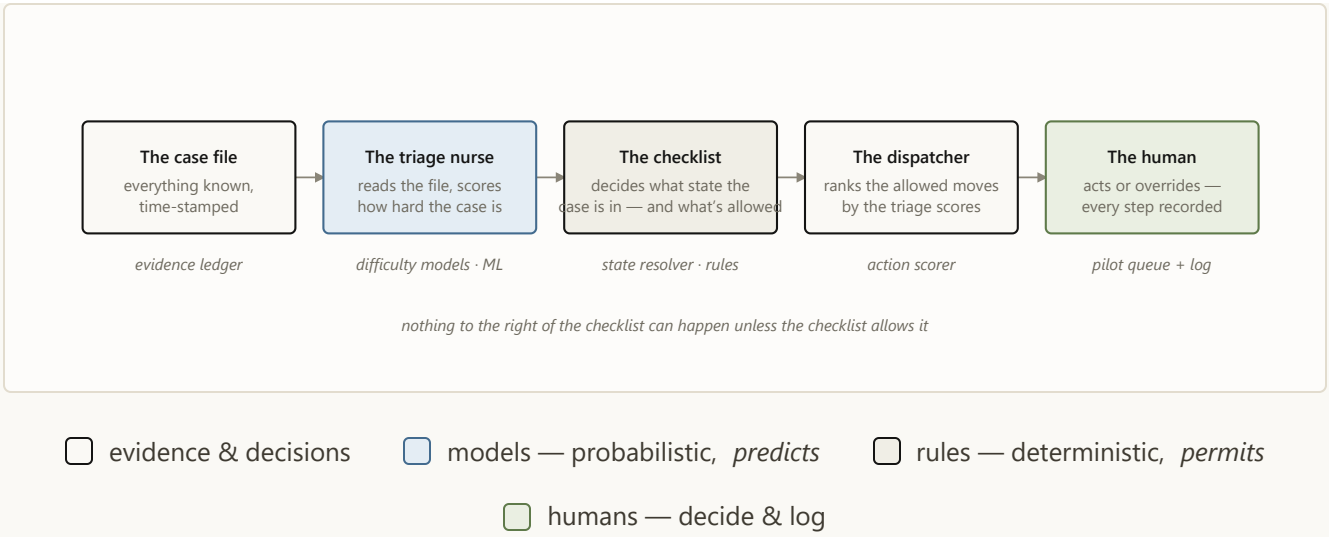


FIGURE 1 The cast, in the order they speak. The color code holds for every diagram below: blue predicts, oat permits, moss decides.

1 The curtain: what the system is allowed to know

The single most important idea in the architecture is not a model — it's a rule about time. The system never looks at a case as one blob of history. It looks at the case *as of a specific morning*, and it may only use facts from **before** that morning. One case therefore becomes many training rows — one per day it stayed open — and our twelve months of history becomes 2.75 million of these `(case, day)` snapshots.

Drag the day below and watch what case 4127 looks like from inside the system. Everything right of the curtain hasn't happened yet — as far as the machine knows, it never will.

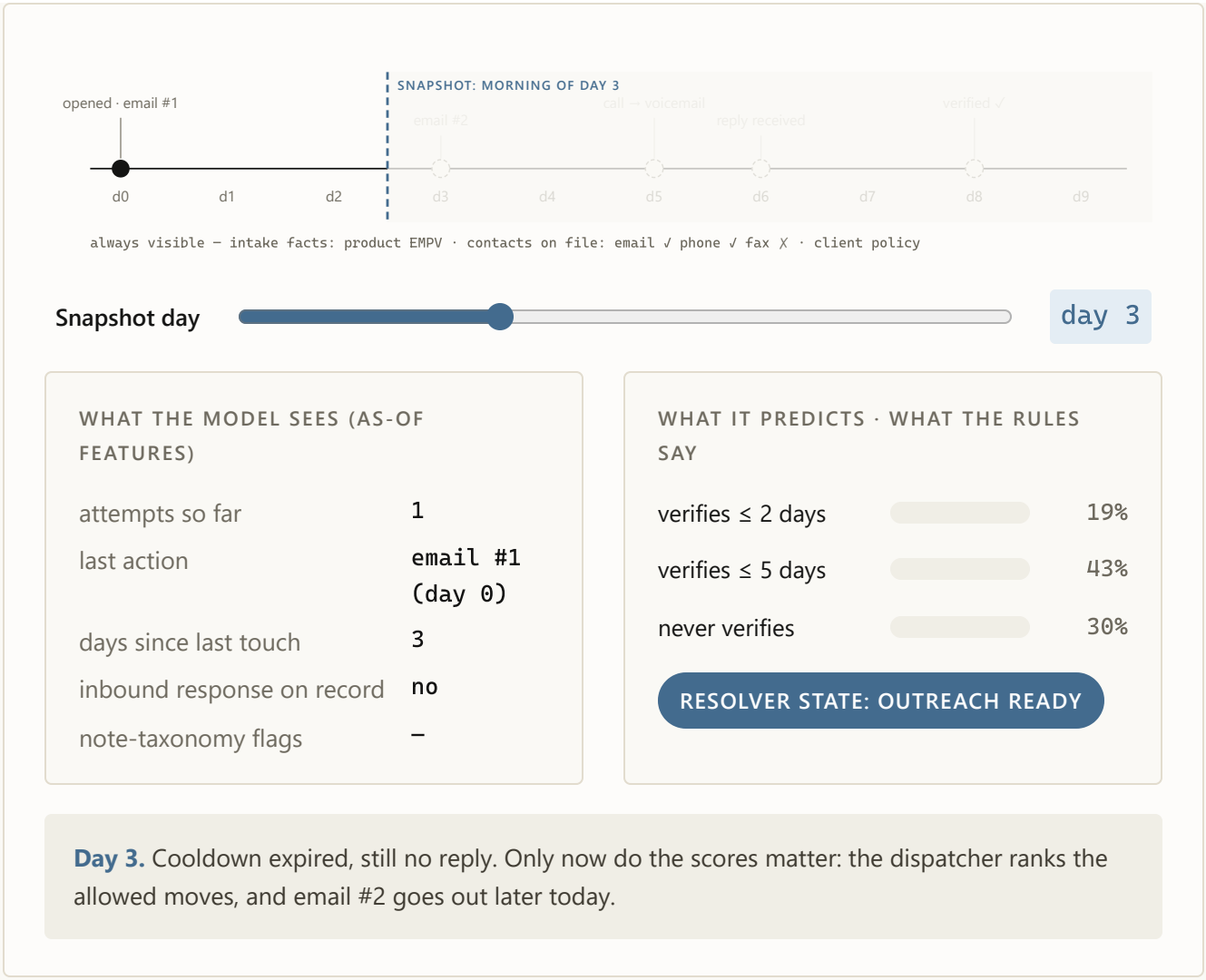


FIGURE 2 The as-of curtain, interactively. Probabilities are illustrative. The curtain is enforced, not assumed: an automated audit re-checks it on every new training matrix before any model trains.

2 Triage: three questions about the same case

Each morning, three calibrated models read the visible half of the file and score it: *will this verify within 2 days? within 5? will it ever?* Calibrated means the numbers behave like frequencies — across all cases scored "30% never-verify," about 30% actually never verify. That property is what makes the scores safe to build queues and thresholds on.

One case's scores are mildly interesting. The population view is where triage earns its keep: sorted by predicted never-verify risk, the easiest tenth of cases fails 3% of the time and the hardest tenth 48%. Same process, same operators — a 16× spread in what was ever achievable. That spread powers the queue (work the winnable middle first) and a fairer scoreboard (measure misses against what was possible, not against a blended average).

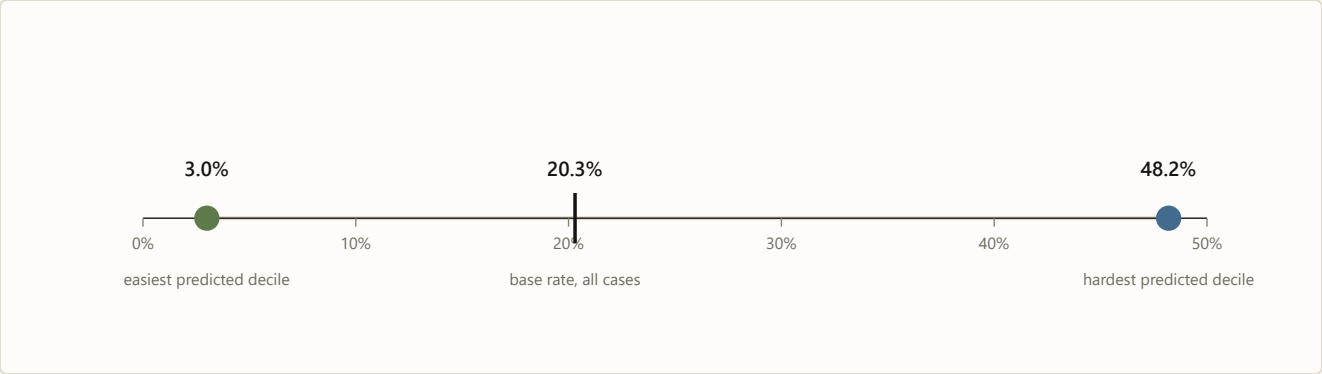


FIGURE 3 Real numbers (May 2026 holdout): observed never-verify rate by predicted-risk decile. Triage doesn't change any single case; it changes which cases get the attention.

3 The checklist: what state is this case actually in?

Before anyone acts on a score, a deterministic resolver asks a fixed sequence of questions any good supervisor would ask — and the first "yes" decides the case's state for the day. The real resolver distinguishes thirteen states; the intuition fits in seven questions:

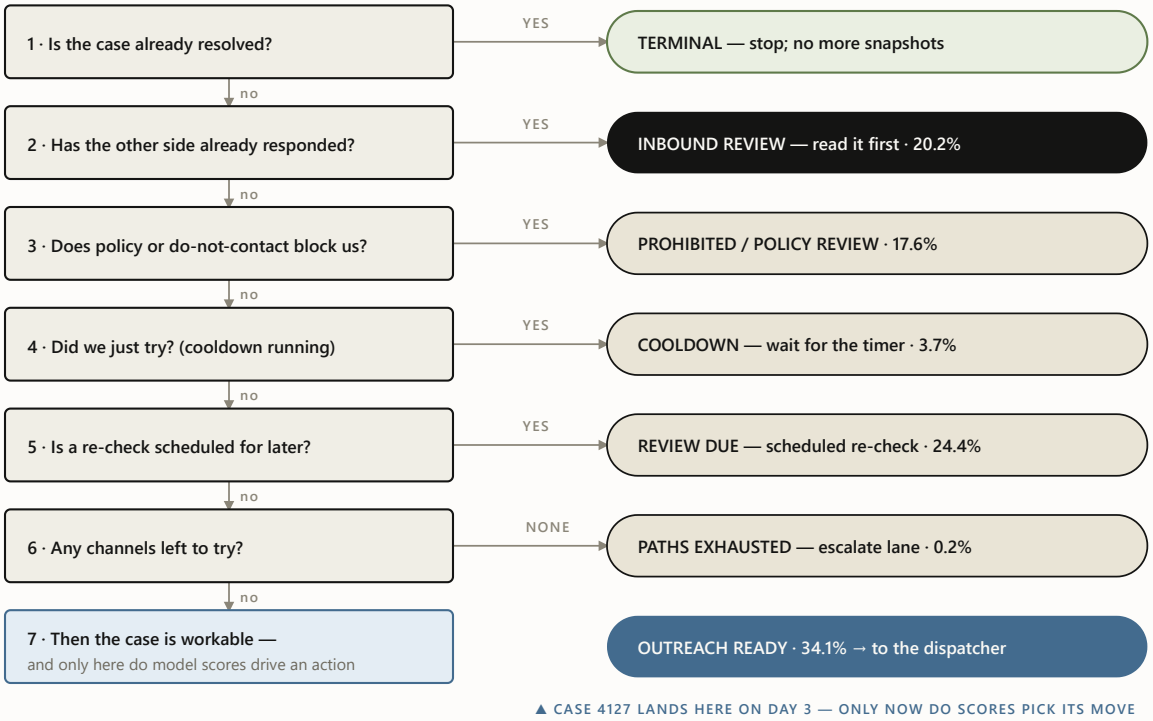


FIGURE 4 The resolver as a supervisor's checklist (a seven-question simplification of the real thirteen states; percentages are each state's share of all 2.75M case-days). Notice question 2: a fifth of all case-days already have a response on record. Answering it deterministically is what caught the ~1-in-7 cases the old flow would have re-contacted. Across all 2.75M case-days: zero do-not-contact violations, zero policy violations — by construction, since scores are never consulted until row 7.

4 The bouncer and the dispatcher: six cards, one recommendation

Suppose it's day 3 and case 4127 landed on "outreach ready." The system now deals six possible moves — the same six for every case, 16.5 million candidate cards in the full run. Eligibility rules physically remove cards before any ranking happens; the model's opinion of a blocked card is irrelevant, because the card is no longer on the table.

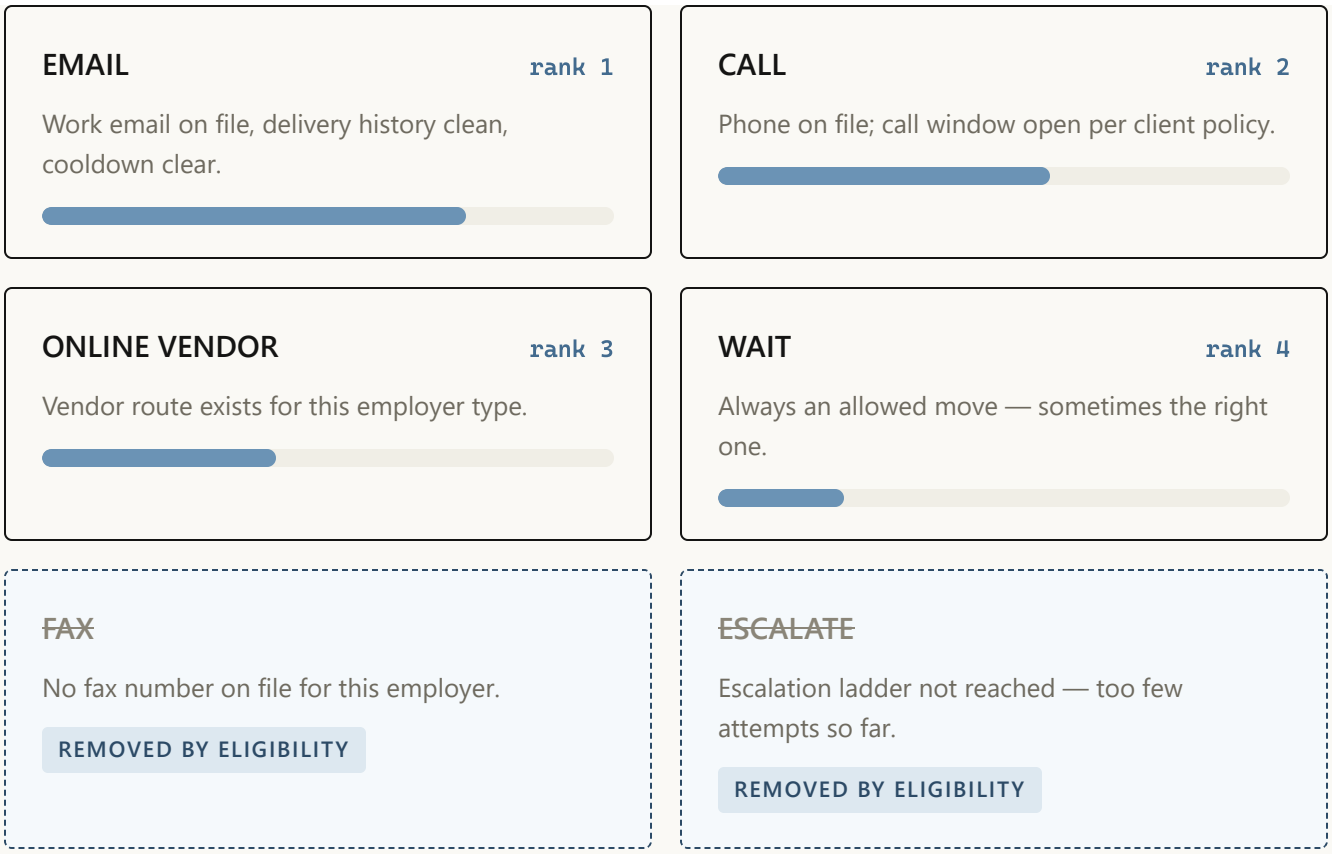


FIGURE 5 Case 4127's hand on day 3. Priority bars are illustrative; ranking reflects what historically moved similar cases under current policy — an observational signal, deliberately not a claim that any channel *causes* better outcomes. Blocked cards never reach the ranker: eligibility is a filter, not a penalty.

The dispatcher's output for the day is exactly one line: *recommended next move: EMAIL, rank 1 of 4 allowed*. In the full backtest, every single case-group ended with an available move — zero "rank-1 but blocked" contradictions, zero empty hands.

5 The flight recorder: a person decides, everything is written down

The recommendation lands in a queue in front of a person, who can take it or override it — and the override is as valuable a signal as the action. Five event types get logged, and together they form the case's complete decision story:

day 3 · 09:12	RECOMMENDED EMAIL — rank 1 of 4 allowed moves, shown to operator
day 3 · 09:14	ACTION EMAIL sent, no override · template family `follow_up` v3
day 4 · 07:40	DELIVERY delivered — no bounce
day 6 · 15:03	REPLY inbound response recorded → resolver flips to INBOUND REVIEW
day 8 · 11:26	OUTCOME verified — outcome joined back to every prior event

FIGURE 6 Case 4127's flight record (fictional timestamps). This chain — recommendation, action or override, template version, delivery and reply, outcome — is the price of ever saying the word "because." Until it's collected at scale, the system reports correlations as observations and nothing more.

6 The heartbeat: how the machine stays honest month over month

Everything above is the *daily* loop. Wrapped around it is a monthly one: new data lands, the snapshot matrix is rebuilt behind the curtain, an automated leakage audit checks the curtain held, challenger models are trained, and a mechanical gate decides — champion or challenger — with the verdict recorded either way. The gate has said no to three of my own improvements this year. That's the feature, not the bug.

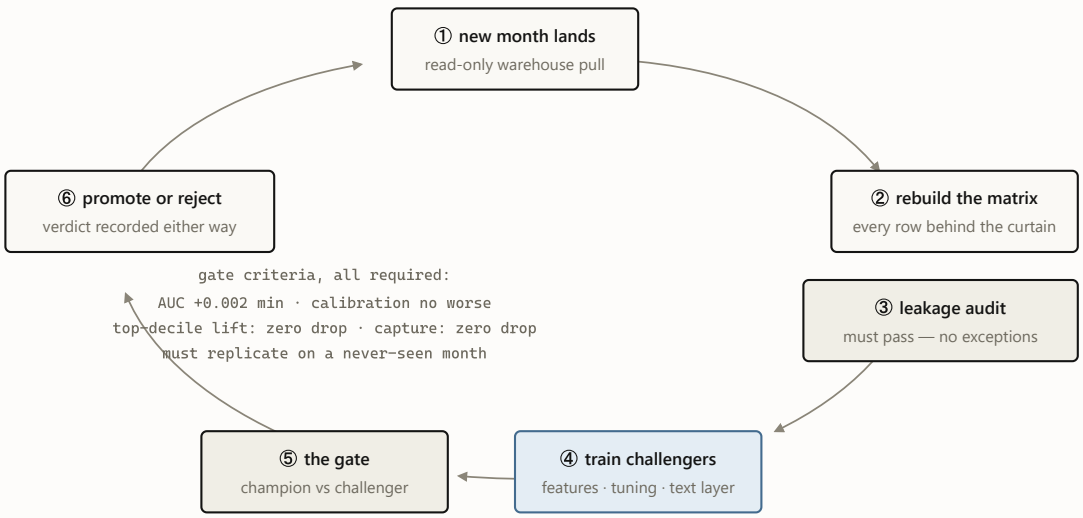


FIGURE 7 The monthly heartbeat. July 2026's cycle promoted text-aware champions for both success targets and rejected the same idea for the never-verify target — its third rejection this year, each one recorded in the champion manifest. A gate that never says no is a rubber stamp.

WHY TWO LANES AND NOT ONE BIG MODEL?

Because the two lanes fail differently. If triage is wrong, an operator works cases in a slightly worse order — cheap, recoverable, invisible. If "what's allowed" is wrong, someone gets contacted who must never be contacted — an incident with a name on it. Splitting the lanes means the expensive failure mode is handled by the component you can audit line by line, and the learning component is confined to the failure mode you can afford. A single end-to-end model would blend both failure modes into one — and price everything at the expensive one.

The whole machine on one page

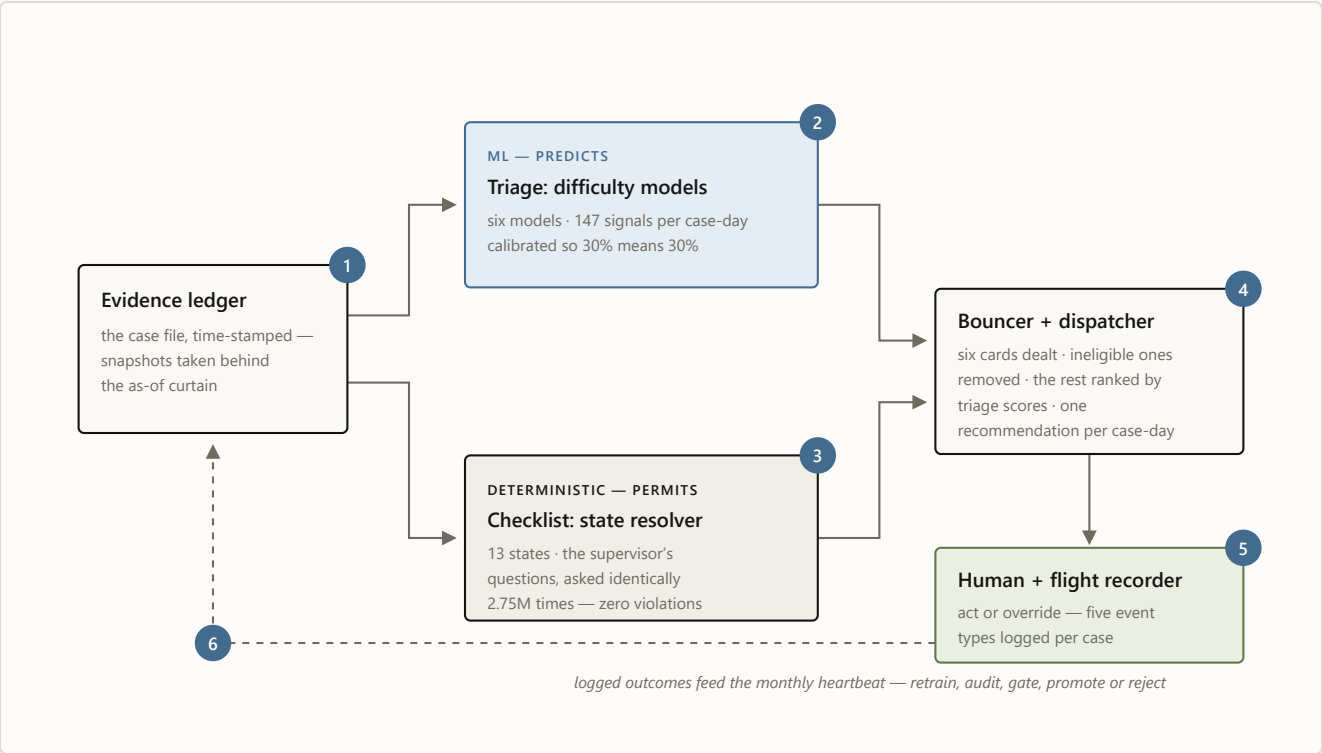


FIGURE 8 The assembled machine, with case 4127's six stops numbered in order: **1** the file is snapshotted behind the curtain · **2** triage scores it · **3** the checklist names its state · **4** the bouncer deals and the dispatcher ranks · **5** a person decides and the recorder writes it down · **6** the logs feed the monthly heartbeat. Blue predicts; oat permits; moss decides.

That's the whole machine. No step is mysterious on its own — a curtain, three scores, a checklist, a bouncer, a queue, a log — and that is deliberate. Each layer is simple enough to explain to the people whose work it touches, and the intelligence lives in how strictly the layers are allowed to talk to each other. Two layers remain on the drawing board, on purpose: once the flight recorder holds enough history, a small language model will draft the communications themselves — the emails, faxes, notes, and call scripts — and an AI contact researcher will hunt fresh, verified emails, fax numbers, and phone numbers for the cases stuck on dead ones. ◆

The series: Building a decision engine for verification casework (the engineering deep dive) · Earned autonomy (the design philosophy) · this illustrated guide · plus the executive briefing and the Murphy Brown field report.

Case 4127 is fictional and its probabilities, ranks, and timestamps are illustrative. Real numbers cited: 2,751,900 case-day snapshots; 16.5M candidate actions; state-mix percentages from the 2026-06-19 resolver run; the 3.0% / 48.2% decile spread from the May 2026 holdout; zero DNC and policy violations across the full backtest. Correlations are observational — no channel or wording is claimed to cause outcomes. Sources in-repo: `README.md`, `presentations/ARCHITECTURE.md`, `docs/ADR/0001`, `config/model_champion_v1.json`.

UBS Verifications · Decision Engine · July 2026

Every verification case ends one of two ways: verified, or *unable to verify*. In the twelve months ending May 2026, UBS worked 876,246 verification searches across four products — employment, education, reference, and license-style checks (EMPV, EDUV, REFV, PLV). Of the 499,465 searches that received at least one attempt, 101,393 ended unable to verify. That UTV rate is the number the business wants down, and it is a frustrating one to manage, because it blends two very different populations: cases that were never going to verify no matter what anyone did, and cases that were winnable but stalled.

The system described here exists to separate those populations and act on the difference. For every open case on every day it answers two questions — *how*

hard is this case? and *what is the next sensible move?* — and it answers them with a strict division of labor: machine-learned models predict, deterministic rules permit. This post explains how the pieces fit, what the evaluation discipline looks like, and the places the discipline said no.

One roadmap note up front. What ships today is calibrated tabular models plus a deterministic policy layer. The next phase adds a **small language model for client messaging** — drafting the emails, faxes, notes, and call scripts, and learning from past communication to draft the best next action. It is deliberately sequenced behind the pilot's instrumentation, so its value can be measured rather than asserted. More on that at the end.

The shape of the system

The architecture is five layers, and the arrows matter more than the boxes.

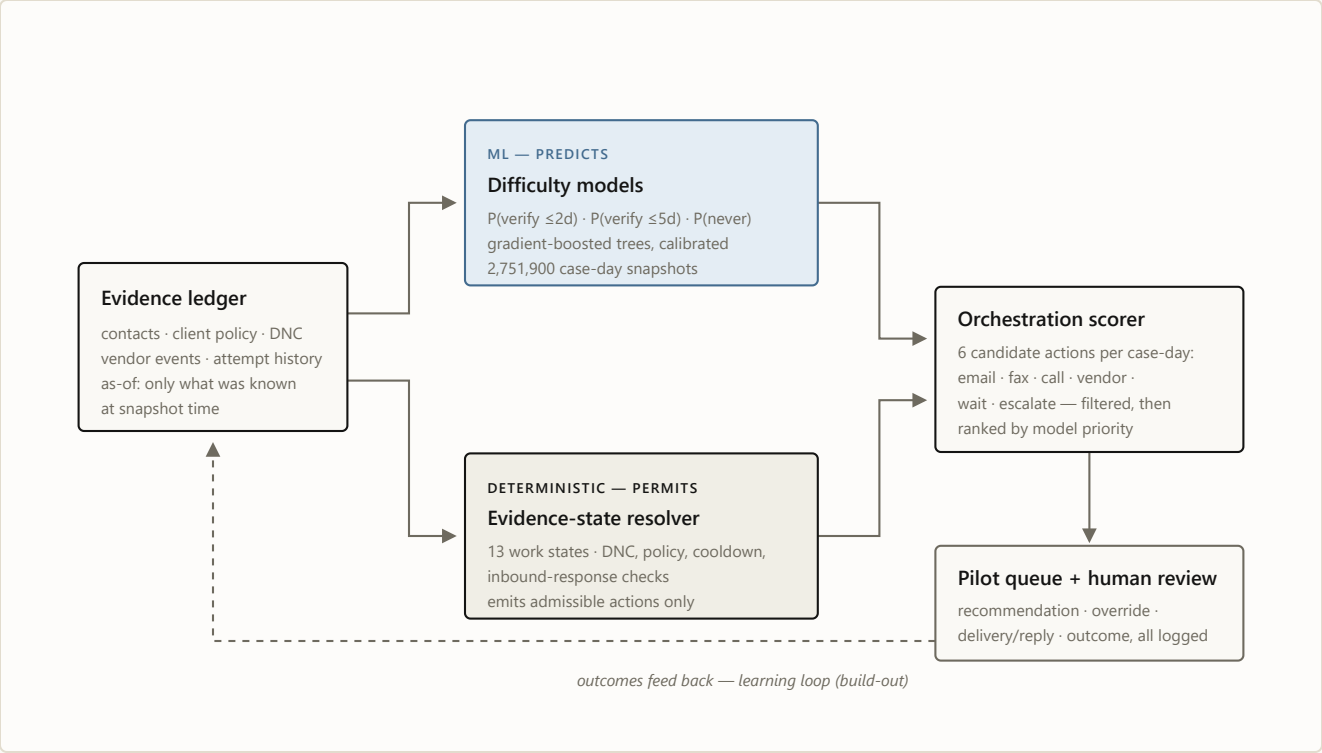


FIGURE 1 The decision path. Models and rules both read the same as-of evidence, but only the resolver decides what is allowed. The scorer ranks within allowed moves; a human works the queue. The dashed loop — learning channel, cadence, and message from logged outcomes — is the build-out, not the current system.

The load-bearing design rule is the one in the middle:

> *Deterministic rules decide what is allowed and what is blocking. Models only prioritize within allowed moves — a model never authorizes an action it cannot justify.*

Do-not-contact, client policy constraints, third-party exclusions, cooldown timers, and "this person already replied" are all resolved by rules, independent

of the models, before any score is consulted. A hard eligibility violation is a filter, not a score penalty a confident model could outvote. In the full 2,751,900-snapshot backtest the resolver produced **zero DNC violations and zero hard-policy violations**, which is only an interesting number because the models never got a chance to create one.

Three questions, one matrix

Three machine-learned models each take one version of "how hard is this case?" — will it verify within two days, within five days, and will it *ever* verify. All three are trained from a single matrix of case-day snapshots: one row per `(searchid, snapshot_day)`, 2,751,900 rows over the twelve-month window, each row restricted to what was knowable at that moment. The models themselves are gradient-boosted decision trees (the HistGradientBoosting family): an ensemble method that builds hundreds of small decision trees, each one trained to correct the mistakes of the trees before it. I chose that family deliberately over deep-learning alternatives — on structured operational data like this it is still the accuracy benchmark to beat; it natively handles the mix of counts, categories, and missing values that case history actually is; it retrains in minutes, which is what makes a monthly self-improvement cycle cheap; and its decisions can be interrogated feature by feature. On top of each model sits a calibration layer that turns raw scores into honest probabilities — when the system says 30%, it happens about 30% of the time — because the consumers of these scores are queues and thresholds, and a queue built on miscalibrated probabilities is a queue that lies about its own workload.

Table 1. Champion models (v3, promoted 2026-07-02). May 2026 is the 299,317-row holdout gate month; June 2026 time month scored with frozen bundles.

TARGET	CHAMPION VARIANT	BASE RATE	ROC-AUC, MAY	ROC-AUC, JUNE O
Never verifies (final UTV) the headline target	baseline, bounded features kept — 3 challengers rejected	0.201	0.706	champion held; lift 2.61× vs 2.4
Verifies within 5 days	note-aware, identity-safe promoted	0.477	0.717	0.7
Verifies within 2 days	note-aware promoted	0.264	0.747	0.7

In plain terms: hand the never-verify model any two open cases — one that eventually verified, one that never did — and it puts them in the right order **71% of the time**, where a coin flip manages 50% (that is what ROC-AUC 0.706 means; every score here reads the same way). The two-day and five-day models sort at 75% and 72%. At day 0 — before any work has happened — a walk-forward, identity-safe variant of the never-verify model reaches **≈81%** across three forward folds (82% / 80% / 81%); knowing on intake which cases probably won't come back is the single most useful thing the system does. The gap between 81% and the pooled 71% is survivorship, not decay: fresh cases differ enormously and are easy to tell apart, while the pool of still-open cases grows more alike as the easy ones resolve out — ranking the survivors is intrinsically harder. And the three champions are not alone: **six learned models run in orchestration** — these three forecasts, the day-0 intake read, day-1/2 re-scores for aging cases, and an early-warning model — all reading the same morning snapshot, all feeding one daily decision pass. What

that buys operationally is the spread: the tenth of cases the model ranks riskiest actually fails **48% of the time against a department-wide base rate of 20%** — 2.4× the true never-verifies packed into the first list an operator works:

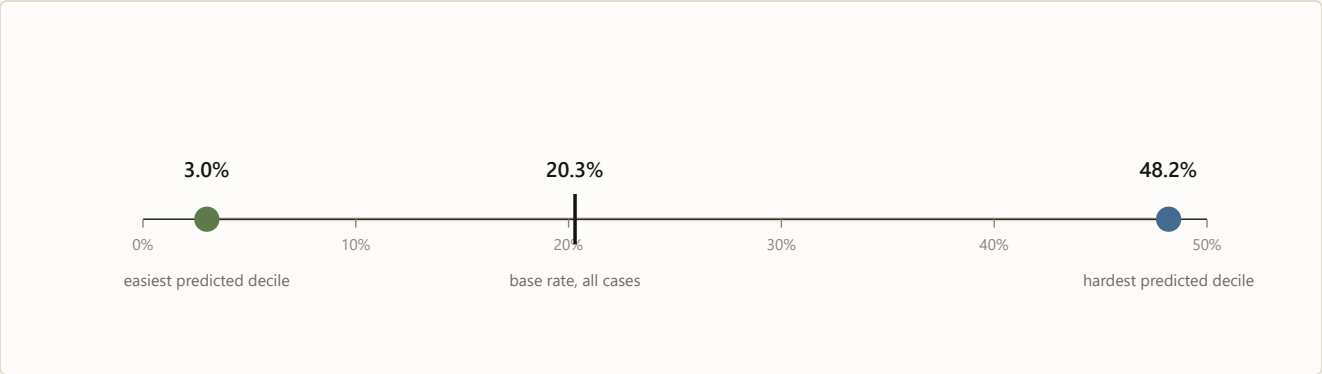


FIGURE 2 Observed unable-to-verify rate by predicted-risk decile, May 2026 holdout. The model separates cases that fail 3% of the time from cases that fail 48% of the time. That spread is what makes a *fair* UTV metric possible: misses can finally be measured against what was actually winnable.

That 16× spread is also the business story. Today's UTV rate silently blames operators for cases that were structurally dead on arrival. Score-conditioned targets flip that: the easy decile is expected to verify almost always, the hard decile almost never, and management attention goes to the winnable middle.

The three champions are also just the tip of the fleet. Six learned models run in this system's production posture, each with its own validation story:

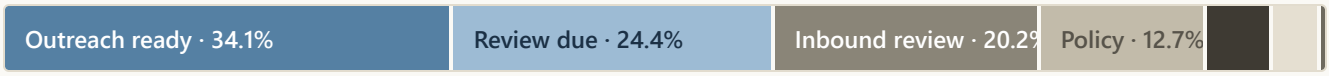
Table 2. The model fleet. All are gradient-boosted trees, calibrated so their probabilities read true (a 30% means 30%), trained under the same as-of matrix discipline; every model passes the leakage audit, and champions additionally clear the promotion gate.

MODEL	PREDICTS	ROC-AUC	VALIDATION	STATUS
Never-verify champion	will the case ever verify	0.706	May holdout + June OOT	champion — bounded

MODEL	PREDICTS	ROC-AUC	VALIDATION	STATUS
Verify within 2 days	resolves ≤2d of snapshot	0.747	May holdout + June OOT	champion — text features
Verify within 5 days	resolves ≤5d of snapshot	0.717	May holdout + June OOT	champion — text, identity-safe
Day-0 intake read	never-verify, at arrival	~0.81	walk-forward, 3 forward months	identity-safe variant
Day-1/2 re-score	re-reads aging cases	0.73 / 0.70	holdout	lifts aging EMPV read 0.62 → 0.71
Early warning	will strand by day 20	0.776	single-month holdout	preliminary

Rules decide what's allowed; models decide what's next

The deterministic half is an evidence-state resolver that classifies every case-day into one of thirteen work states — terminal, already-responded, policy-blocked, awaiting response, cooldown, review-due, research-needed, digital-paths-exhausted, standard-paths-exhausted, outreach-ready, and so on — and derives the set of admissible actions from the state, never from a score. Here is where the 2.75M snapshots actually sit:



Outreach execution ready	34.1%	Review due (scheduled re-check)	24.4%
Inbound review required	20.2%	Policy / DNC review required	12.7%
Outreach prohibited	4.9%	Cooldown active	3.7%
Digital / standard paths exhausted	0.2%		

FIGURE 3 Resolver state mix across 2,751,900 snapshots (2026-06-19 ladder run). The interesting segment is the third one: cases where a vendor response is already on record but the legacy flow would have kept contacting.

That third segment hides the most immediately bankable finding in the project. On roughly **401,000 case-days — about one in seven** — a vendor-channel response had already been received, yet the process as practiced would have actively re-contacted the party. The resolver routes every one of them to review-first instead. No model was needed for this; it fell out of writing down, deterministically, what "already responded" means. (The signal is a vendor receipt, not parsed reply content, and it doesn't exist for every product — the honest version of this number lives in the repo docs alongside its caveats.)

Downstream of the resolver, the scorer builds six candidate actions per case-day — email, fax, call, online vendor, wait, escalate — 16,511,400 candidate rows in the full run, filters them through eligibility, and ranks the survivors by the calibrated scores. A seven-stage pipeline chains candidates → eligibility → scoring → call-escalation ladder → wait policy → workflow router → pilot queue; every stage checks its own output and halts the run on any failure, and a strict contract validator signs off at the end. The full run produced zero blocked rank-1 actions, zero no-action groups, and a queue where every group has an available move.

No peeking, by construction

The easiest way for a model like this to look brilliant is to let information from after the outcome leak into its features — post-outcome status fields, completion timestamps, identity proxies. Every feature here is built under strict "as-of" discipline instead: 117 columns, each computed with strictly-before semantics, so a case scored on a Tuesday morning is scored only on what was knowable that Tuesday morning. An automated leakage audit re-

checks that boundary on every new training matrix before any model is allowed to train on it — and runs again at every monthly retrain. The scores in this post are conservative by design, measured the hard, honest way.

Promotion is a gate, not a debate

Model improvements are promoted per target — there is no single "champion model," there are three — and promotion is mechanical. A challenger must clear every criterion against the sitting incumbent; a single checked-in champion config is the source of truth, and rejections are machine-recorded in the champion manifest.

ROC-AUC	at least +0.002 over the incumbent, with bootstrap confidence intervals excluding zero
Brier score	no worse than the incumbent — calibration is not negotiable
Top-decile lift	zero drop tolerated — the top of the queue is what operators actually work
Capture at top 10%	zero drop tolerated
Slice gate	per-product and per-day slices, zero-tolerance with a reviewed, unit-tested allowlist for false positives
Out-of-time replication	the win must survive a never-seen month scored with frozen bundles

The gate earns its keep on what it rejects. Three times this year I tried to improve the never-verify model — a settings-tuning pass, a guarded version of

the same sweep, and this cycle's note-reading variant — and the gate turned all three away **for the same reason**: each was slightly better on average across the whole book, and slightly worse at the very top of the ranked worklist — the first page operators actually work, where accuracy is worth the most. One rejection is bad luck; three identical ones is a message. For this target, average accuracy and top-of-list sharpness genuinely pull against each other, so the next attempt changes the training data itself — June's outcomes finish maturing in the July pull — rather than pushing a fourth model variant.

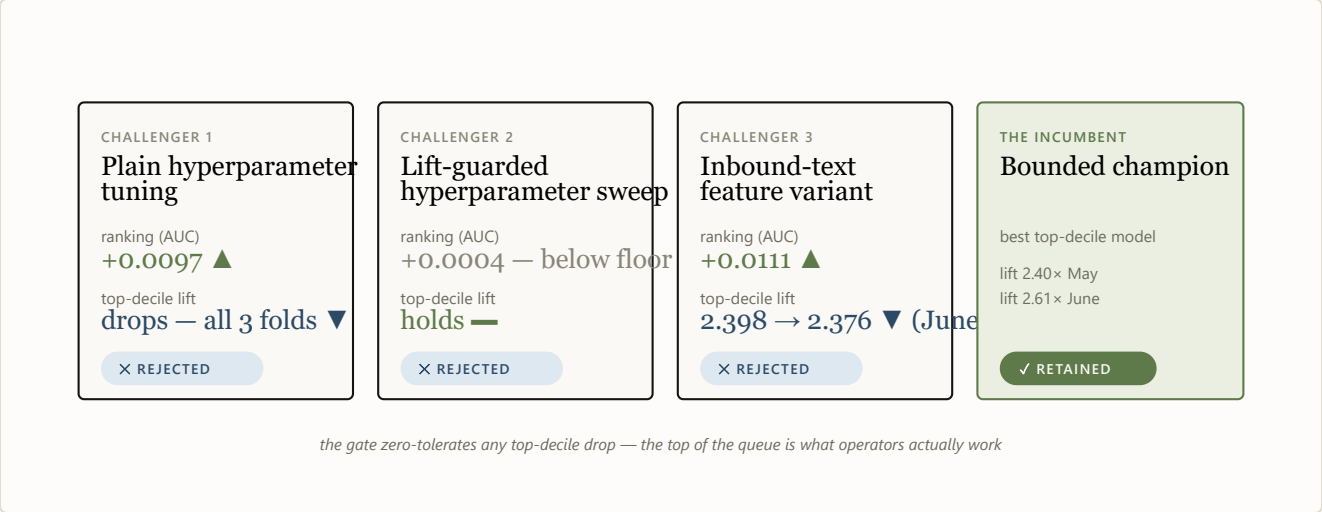


FIGURE 4 The gate at work on the never-verify target: three challenger classes, three different levers, one identical failure mode. Each verdict is machine-recorded in the champion manifest — the rejections are as documented as the promotions.

A gate that never says no is a rubber stamp. Ours says no to us regularly, in writing.

Reading the notes without keeping them

The biggest accuracy gain this cycle came from the data source I had been most careful to avoid: the free-text notes operators and vendors leave on case

history (“left a voicemail”; “reached HR, call back Tuesday”). Notes are where the richest signal lives — and where the risk lives twice over: outcome language (“verified via…”) leaks answers into training, and raw text carries the privacy exposure, so persisting it into training artifacts was never on the table.

The design that threads the needle: **a deterministic reading layer classifies each note the moment it streams past — then throws the text away**. Every note is sorted into eleven kinds of operational signal (contact made, voicemail left, callback promised, redirected to a third party, …), and only the signal survives: which kinds of events a case has seen, how recently, how often. Deterministic matters here — the same note always produces the same labels, so the privacy boundary has no black box in it and the whole layer can be audited and replayed. That turns 17.9 million unreadable notes into 30 new machine-usable signals per case-day for the learned models, computed under the same no-peeking time discipline as everything else; about 28% of case-days carry at least one.

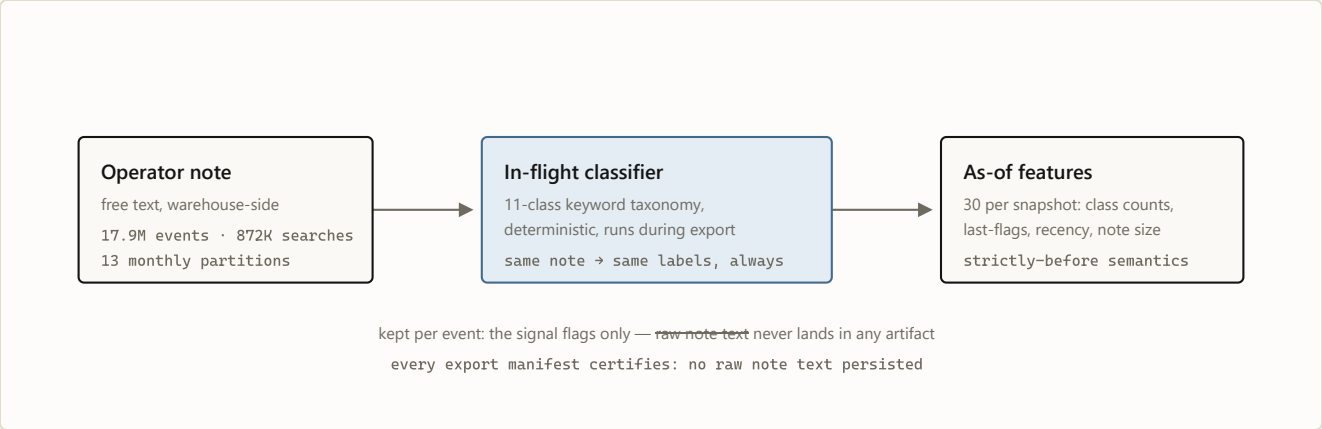


FIGURE 5 The inbound text layer. Signal crosses the boundary; text does not.

The payoff cleared the full promotion gate for both success targets — the five-day model improved on every metric with confidence intervals excluding zero, and the gain *grew* on the never-seen month; the two-day model held the same

pattern. The never-verify variant, as covered above, gained on average, lost the top decile, and died at the gate.

One operational note survives from that build: a routine row-count check against the export manifests caught a corrupted export *before anything consumed it*. Cheap invariant checks between pipeline stages are the highest-ROI code in the system.

What I deliberately don't claim

The repo maintains an explicit disallowed-claims list, and it does more for the project's credibility than any AUC. The current lines:

- **No causal claims about channel, wording, or templates.** Historical patterns like "email-first cases verified more often than call-first cases" are observational — selection effects all the way down. Causal language stays banned until the pilot logs the full chain for each case: recommendation shown, action taken or override reason, template version, delivery/reply, final outcome. The logging schema for exactly those five event types is built and validated; the evidence is not yet collected.
- **Not "identity-free," only "direct identifiers stripped."** Action-facing models exclude direct agent and team IDs behind a drift gate, but behavioral agent attributes remain. Identity-bearing champions stay `provisional` for action-facing use, and the resolver applies no-direct-identity fallbacks automatically until that gate approves.
- **Decision support today — automation is earned, not assumed.** Nothing auto-sends yet and no decision is finalized by a machine; the

wiring for automation exists, and each step toward it — exploration, auto-drafts, then gated auto-send — unlocks on pilot evidence, with exploration shipping disabled by default. The router is shadow-review metadata, not an autonomous policy — and I won't train a "learned policy" on historical action logs and pretend the logs were experiments.

- **The language layer is next-phase, on purpose.** The endgame — a small language model that drafts the emails, faxes, notes, and call scripts, learning from past communication to draft the best next action — is sequenced last because without the causal instrumentation above no one could tell whether it works.
- **No rework-savings claims — the QA loop is measured, not yet moved.** A reproducible July baseline timed the quality-review loop (education + employment, Jan–Jun 2026): a returned check runs a median 11.2 days from first QA return to approval, against 5.0 days for a typical check's entire first pass — and 85.6% of coded return reasons are process-and-judgment classes, the ones the resolver, eligibility rules and lanes exist to prevent. "Exist to prevent" is a design statement, not an effect size: the effect claim waits for the pilot, and hands-on handle time waits for work-item state logs that nothing emits today.

What's next

Three threads, in order of readiness. The **July cycle** re-runs the monthly spine — the text exporter and matrix builders are month-partitioned and resumable for exactly this — and matures June's never-verify labels, which is the most promising lever left for the one target that keeps rejecting challengers. The **identity proxy gate** either approves the identity-bearing

champions for action-facing use or keeps the fallbacks in place; the walk-forward drift harness exists, and the decision is deliberately not mine to hand-wave. And the **play engine** — recommending channel + timing + message family + template as a unit, then eventually learning that choice from pilot data that records which options were offered, not just which was taken — waits on the only thing that can unlock it: real logged outcomes from humans working the queue.

One more build sits on the same roadmap: a **contact-researcher loop** — an AI agent that goes looking for better contact paths (fresh, verified emails, fax numbers, and phone numbers for employers, registrars, and vendors) and hands them to the eligibility layer as an upgraded contact plan. Today a case lives or dies on the contacts already in its file; a researcher that replaces a dead fax line before the third failed attempt turns unworkable cases workable. Like every other layer, its suggestions enter as candidates for the rules to admit — never as automatic dial targets.

And after the play engine comes the final step: a **small language model that writes the communications themselves** — the emails, the faxes, the case notes, and the scripts our call operators work from — learning from the logged record of past communication and its outcomes to draft the best next action for every case. It is sequenced last because it is graded last: only the pilot's causal instrumentation can tell a well-worded email from a lucky one.

The system today is, on purpose, the boring version: calibrated probabilities, deterministic guardrails, mechanical promotion gates, and a written list of things I refuse to say. Every flashier layer gets built on top of that — or not at all. ◆

Reproducing this. From the repo root: `py -3.12 code\validate_repo.py` (canonical gate: compile, tests, configs) · `py -3.12 code\run_pipeline.py` (seven-stage decision path) · `py -3.12 code\validate_decision_contract.py strict` · `py -3.12 code\pilot_logging.py validate`.

The series. This is the engineering deep dive. Also: Earned autonomy (the design philosophy) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

Internal engineering note written in the style of a public post. All figures are backtested and as-of; nothing here claims pilot results or causal channel/template effects. Sources of record in-repo: `README.md`, `docs/TUNED_CHAMPION_PROMOTION_20260702.md`, `docs/INBOUND_TEXT_PROMOTION_20260702.md`, `docs/JUNE_2026_OOT_VALIDATION.md`, `config/model_champion_v1.json`, `presentations/`.

UBS Verifications · Decision Engine · July 2026

There is a class of problem that looks made for AI and punishes naive AI harder than almost anything else. Call it *operational casework*: hundreds of thousands of open cases, each needing a next move today; hard compliance constraints — do-not-contact lists, client policy, regulated channels; human operators who own the outcomes; and labels that arrive late, noisy, and entangled with the process that produced them. Verification work is our instance — roughly 880,000 cases in a year, a fifth of the attempted ones ending “unable to verify” — but collections, claims, case management, and outreach operations all share the shape.

The tempting move is the end-to-end agent: feed it the case, let it decide, act, and learn. In this problem class that design fails in a specific, predictable order. First it violates a constraint, because a model treats policy as a feature with a weight rather than a boundary — and at our volume, a 99.9%-compliant model is thousands of violations. Then it flatters itself, because historical logs reward whatever the process already did. And then it can't be trusted with the interesting decisions, because nothing in its training data identifies what would have happened under a different action. Our own verification bot is the live case study: it runs without this operating layer, and the field report on what it leaves behind — ungated pickup, note-only handoffs, reopening closures — is this essay's companion piece.

So I built the opposite of an end-to-end agent, and the interesting part isn't the system — it's the discipline. Four rules, in increasing order of how much they cost us.

1. Split the decision

Every "what should we do next?" question decomposes into two questions with different failure tolerances. *What is permitted?* has a tolerance of zero: getting it wrong is an incident, and the answer must be explainable after the fact, case by case. *What is preferred?* has a tolerance of "be usefully better than the status quo": getting it wrong wastes effort. The pattern's first rule is to never let one component answer both.

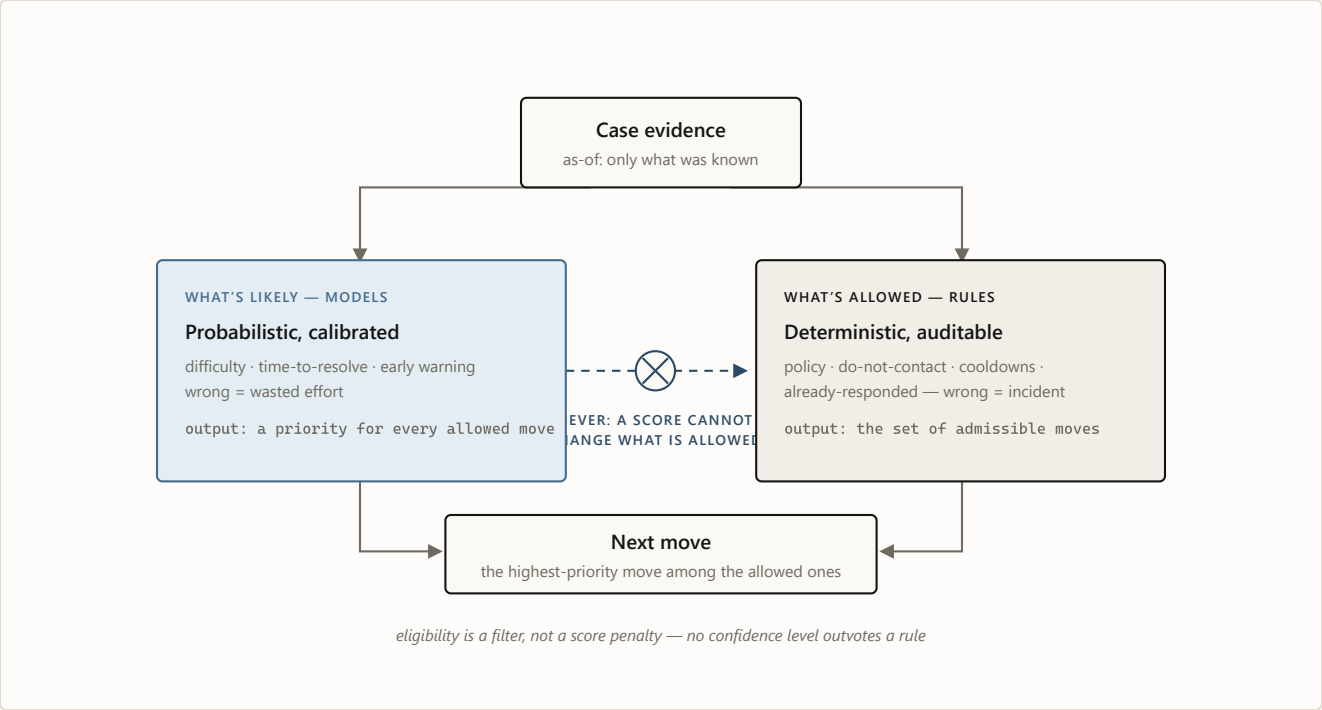


FIGURE 1 The split. Both lanes read the same evidence, but only the deterministic lane can say no. In this system the rules lane resolved 2.75 million case-days into thirteen work states with zero do-not-contact and zero hard-policy violations — an unremarkable number, which is the point: the models never had the opportunity to create one.

The split also produced the cheapest large win in the project, and it came from the *rules* lane. Writing down, deterministically, what "this person already responded" means revealed that on roughly one case-day in seven, a response was already on record while the process as practiced would have kept contacting. No model, no training run — just a definition, applied consistently at scale.

2. Forecast first, act later

The second rule is about sequencing: the first machine-learned product should be a *forecast*, not an action. Forecasts pay rent under zero autonomy. A calibrated estimate of "this case will never verify" is immediately useful for

triage — work the winnable middle, not the doomed tail — for early warning, and for something subtler that turned out to matter most to the business: **fixing the metric itself**.

An aggregate failure rate blames operators for cases that were structurally dead on arrival. A difficulty model unblinds that: the easiest predicted decile fails 3% of the time, the hardest 48%. Once you can condition on difficulty, "how are we doing?" becomes "how are we doing *against what was achievable?*" — a fair scoreboard, and a fundamentally different management conversation. The model changed what the organization measures before it changed anything the organization does.

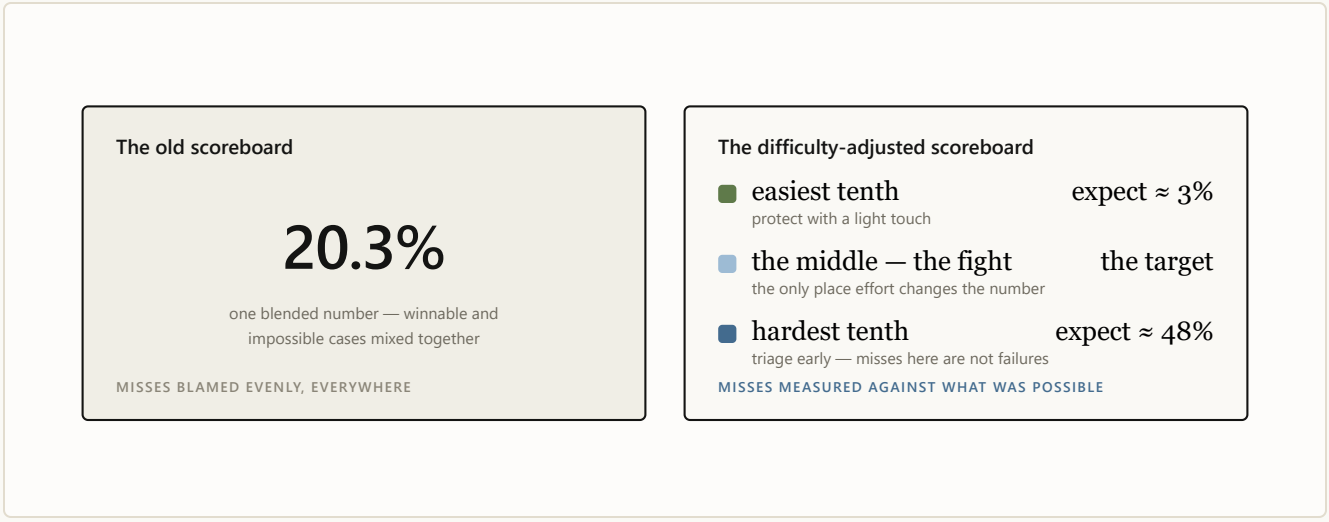


FIGURE 2 Same cases, two scoreboards. The blended number can only go up or down; the difficulty-adjusted one says where to spend effort and whom to stop blaming. This reframe — not any single prediction — is the business product of the forecast layer.

Forecast-first has a quiet second benefit: it forces the data discipline that everything later depends on — strictly as-of features, automated leakage audits, calibration checks — while the stakes are still just a number on a validation report. If the first product were an acting agent instead of a forecast,

a data problem would arrive as behavior in production; in a forecast layer it arrives as a number you can catch and fix before anything acts on it.

Restraint, to be clear, is not absence. Inside this pattern run six learned models over a 147-signal read of every case-day, retrained monthly under audit. The discipline is not a substitute for the machine learning — it is what makes that much machine learning safe to run.

3. Price every claim

Most ML systems fail socially before they fail technically: they say more than they know, someone believes it, and the credibility never recovers. The third rule is to treat claims as things with a price — and to refuse to make any claim whose instrumentation you haven't built.

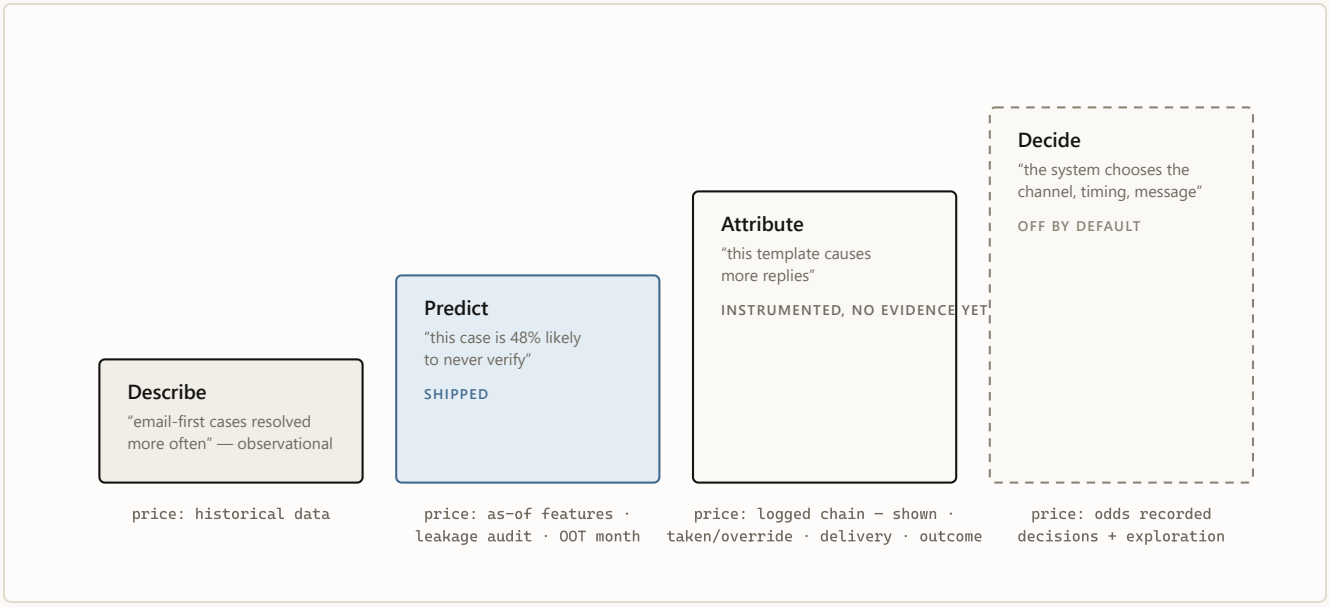


FIGURE 3 The ladder of claims. Each rung is priced in instrumentation, and the price is paid before the claim is made, not after. I hold the line publicly at rung two: correlations like "email-first resolves more often" are reported as observation, never as advice, because selection effects — not wording — may be doing all the work.

Two artifacts make this rule real rather than aspirational. The first is a written **disallowed-claims list** in the architecture record — no causal channel or template claims, no calling the router an autonomous policy, no training a "learned policy" on historical logs and pretending the logs were experiments. The second is a **mechanical promotion gate** for the models themselves: a challenger must beat the champion on ranking quality without giving up any top-of-queue sharpness or calibration, with confidence intervals excluding zero, replicated on a never-seen month. That gate has rejected three consecutive improvements to the headline model — each better on average, each worse where operators actually work. Three rejections in writing is the system functioning, not failing.

Autonomy is not a capability you install. It is a permission the evidence grants — and the default answer is no.

---

4. Let the system earn its next layer

The final rule turns the other three into a trajectory. Authority is granted in stages, each stage gated on evidence the previous stage was designed to produce — and every gate defaults to closed.

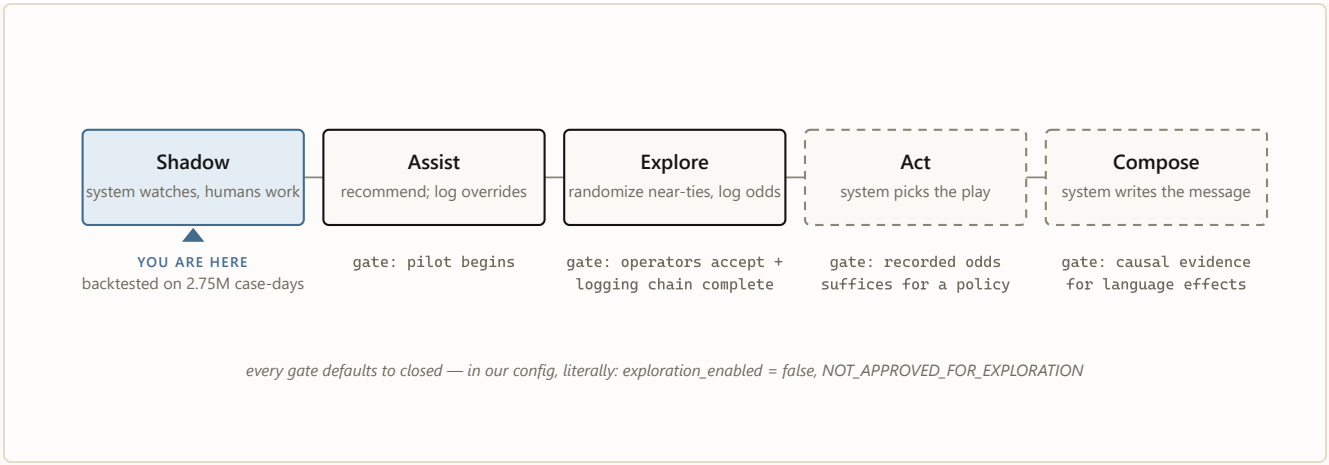


FIGURE 4 The autonomy ladder. Each stage exists to generate the evidence the next stage needs: the pilot logs overrides, overrides calibrate trust, exploration records each option's odds of being offered, and those recorded odds make a learned policy honest. Skipping a rung doesn't accelerate the system — it caps how far it can ever be trusted to go.

This is why the system's most capable component is the one that doesn't exist yet — and that's the ladder working as intended. The destination is explicit: a small language model that writes the outreach itself — the emails, faxes, case notes, and the scripts the call team works from — learning from the logged record of past communication and its outcomes to draft the best next action, alongside an AI contact researcher that hunts better contact paths for the cases stuck on dead ones. Message generation is the most capable layer and the least verifiable one — its evidence requirements sit at the very top — so it is sequenced last, behind causal instrumentation that today exists as schemas and validators rather than collected data. The roadmap names the destination; the discipline decides the pace.

What restraint buys

The pattern has real costs, and it pays for them. The ledger, honestly kept:

WHAT IT COSTS	WHAT IT BUYS
Slower headline progress — the gate rejects your own improvements, and rejections don't make launch announcements.	Deployability from day one — a system that provably cannot violate policy can enter a regulated workflow while still learning to be useful.
Boring demos — "calibrated forecast plus rules" never wows a room.	Failures that are cheap and legible — a wrong model means a suboptimal ordering, not an incident with a name on it.
Unglamorous work first — logging schemas, leakage audits, and manifests before any intelligence.	A metric the business can act on — difficulty-adjusted targets outlive any particular model version.
Self-correction on the record — every gate rejection and validation miss is written down, forever.	Credibility that compounds — every recorded "no" is what makes the eventual "yes" believable.

FIGURE 5 The trade, side by side. Everything on the left is real and annoying; everything on the right is why the pattern survives contact with a regulated business.

WHEN NOT TO USE THIS PATTERN

The full apparatus is overkill where its costs buy nothing: actions that are cheap and reversible, no compliance surface, outcomes observable within minutes, no human in the loop to protect. Ranking a newsletter's subject lines needs an A/B test, not an evidence-state resolver. The pattern earns its weight precisely where mistakes are expensive, constraints are regulatory rather than aesthetic, and trust — of operators and regulators — is the scarce resource.

The boring version, on purpose

One level down, this system is pipelines, gates, and calibration curves — the companion post walks through all of it. At this level it is a single idea applied four ways: **separate the question of permission from the question of preference, and make every expansion of the system's authority something the evidence, not the roadmap, decides.** The result is a system whose most advanced feature, for now, is its restraint — and whose path to becoming more than that is written down, priced, and gated in advance. ◆

The series. This essay is the design philosophy. Also: the engineering deep dive (architecture, champions, the promotion gate) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

Internal note written in the style of a public post. All figures are backtested and as-of; correlations are reported as observation, never as causal advice; pilot results are not claimed. Sources of record in-repo: `docs/ADR/0001-verification-play-decision-engine.md`, `README.md`, `config/model_champion_v1.json`, `presentations/`.

UBS Verifications · Decision Engine · July 2026